

# Arsitektur Aplikasi Terstruktur: Menguasai DTO, Request Validation, Error Handling Konsisten, dan Clean Architecture di Go

---

**Durasi:** 3 Jam (180 Menit) **Level:** Intermediate (Menengah)

Ketika membangun aplikasi backend tingkat produksi, menulis seluruh kode program dalam satu file atau mencampuradukkan logika database langsung di dalam HTTP handler adalah kebiasaan buruk yang menyulitkan pengujian (*untestable*) dan pemeliharaan kode (*unmaintainable*). Anda harus memisahkan tanggung jawab kode program menggunakan arsitektur perangkat lunak yang terstruktur seperti **Clean Architecture**, mengamankan payload menggunakan **DTO**, serta menstandarisasi **Error Handling** di seluruh sistem.

Modul ini akan membedah konsep pemisahan data menggunakan DTO, pemisahan lapisan kode (*decoupling layers*) berdasarkan prinsip Clean Architecture, perancangan custom validator, serta standarisasi penanganan error dari database hingga response klien.

---

## Sesi 1: DTO (Data Transfer Object) & Request Validation (45 Menit)

---

### 1. Mengapa Memisahkan DTO dari Model GORM?

**DTO (Data Transfer Object)** adalah objek yang dirancang khusus untuk memindahkan data antara klien dan server (payload request/response). Model GORM adalah objek yang dirancang untuk merepresentasikan skema tabel fisik database. Mencampuradukkan keduanya (misal menggunakan struct GORM langsung sebagai parameter binding di Gin) sangat berbahaya karena:

1. **Mass Assignment Vulnerability:** Klien jahat bisa mengirimkan payload JSON berisi `"role": "admin"` atau `"balance": 9999999` yang tidak sengaja ter-bind langsung ke database jika struct GORM diekspos langsung sebagai input binding.
2. **Decoupling:** Format JSON yang dikomunikasikan dengan klien belum tentu sama persis dengan kolom fisik database. DTO memberikan fleksibilitas untuk memformat ulang data tanpa merusak skema tabel.

## 2. Request Validation dengan `go-playground/validator`

Gin menggunakan pustaka `go-playground/validator` di latar belakang. Kita dapat mendeklarasikan aturan validasi langsung di properti struct DTO menggunakan tag `binding`.

```
type CreateProductDTO struct {
    SKU    string `json:"sku" binding:"required,alphanum,min=5,max=20"` //
    // Alfanumerik, panjang 5-20
    Name   string `json:"name" binding:"required,min=3,max=100"`
    Price  float64 `json:"price" binding:"required,gt=0"` // Harus >
    // 0
    Stock  int    `json:"stock" binding:"required,min=0"` // Harus ≥
    // 0
}
```

## 3. Membuat Custom Validator di Gin

Jika aturan validasi standar tidak mencukupi, kita dapat mendaftarkan fungsi validasi kustom.

- **Contoh Kasus:** Kita ingin memastikan format SKU wajib diawali dengan prefix `"SKU-"`.

```
package main

import (
    "strings"

    "github.com/gin-gonic/gin/binding"
    "github.com/go-playground/validator/v10"
)

// 1. Fungsi Validasi Kustom
var SKUValidator validator.Func = func(fl validator.FieldLevel) bool {
    sku, ok := fl.Field().Interface().(string)
    if ok {
        // Valid jika sku diawali dengan 'SKU-'
        return strings.HasPrefix(sku, "SKU-")
    }
    return false
}

func RegisterValidators() {
    // 2. Daftarkan fungsi ke engine validator bawaan Gin
    if v, ok := binding.Validator.Engine().(*validator.Validate); ok {
        _ = v.RegisterValidation("sku_format", SKUValidator)
    }
}
```

*Penggunaan di Struct DTO:*

```
type CreateProductDTO struct {
    SKU string `json:"sku" binding:"required,sku_format"`
}
```

## Sesi 2: Arsitektur Bersih (Clean Architecture) di Go (45 Menit)

**Clean Architecture** (atau sering disebut Hexagonal / Onion Architecture) bertujuan untuk membuat kode program menjadi mandiri dan tidak ketergantungan pada pustaka luar (*framework independent*), mudah diuji (*testable*), serta mudah diganti basis datanya tanpa mengubah logika bisnis.

```
graph TD
    Delivery[Delivery/Handler Layer - Gin HTTP] -->|Memanggil| Usecase[Usecase/Service Layer - Logika Bisnis]
    Usecase -->|Memanggil| Repository[Repository Layer - Akses GORM]
    Repository -->|Interaksi| DB[(PostgreSQL Database)]
    Domain[Domain/Entities Layer - Interfaces & Core Models] -.→|Menjadi Kontrak Untuk| Usecase
    Domain -.→|Menjadi Kontrak Untuk| Repository
```

### 1. Lapisan Domain (Entities & Interfaces)

Lapisan paling dalam yang menyimpan definisi model data murni bisnis (tanpa tag GORM/Gin) serta interface kontrak fungsi. Lapisan ini tidak boleh mengimpor pustaka eksternal apa pun.

- **Isi:** `product.go` (Struct entity murni & interface repository/usecase).

### 2. Lapisan Repository (Database Implementation)

Lapisan yang bertanggung jawab mengurus interaksi baca-tulis ke database (PostgreSQL via GORM). Lapisan ini mengimplementasikan interface repository yang didefinisikan di lapisan Domain.

- **Isi:** `product_repository.go` (Mengandung perintah `gorm.DB`).

### 3. Lapisan Usecase / Service (Business Logic)

Lapisan yang berisi alur logika bisnis inti aplikasi (validasi bisnis, kalkulasi harga, integrasi transaksi). Lapisan ini memanggil lapisan repository melalui interface (Dependency Inversion).

- **Isi:** `product_usecase.go`.

## 4. Lapisan Delivery / Handler (HTTP Transport Layer)

Lapisan paling luar yang berhadapan langsung dengan dunia luar (klien). Lapisan ini menangani framework routing (Gin), menangkap request body, memvalidasi DTO, memanggil Usecase, dan mengembalikan JSON response.

- Isi: `product_handler.go` (Mengandung `*gin.Context` ).

## Sesi 3: Penanganan Error yang Konsisten (Consistent Error Handling) (45 Menit)

Salah satu tanda aplikasi yang profesional adalah konsistensi respons ketika terjadi kesalahan. Klien tidak boleh menerima *stack trace* mentah dari database (seperti `"pg: unique constraint error"` ), melainkan pesan terstruktur yang mudah dipahami.

### 1. Deklarasi Sentinels Error pada Layer Bisnis

Definisikan error bisnis di layer Domain agar dapat digunakan bersama oleh usecase dan handler:

```
package domain

import "errors"

var (
    ErrProductNotFound = errors.New("produk tidak ditemukan")
    ErrSKUDuplicate    = errors.New("sku produk sudah terdaftar")
    ErrInternalServerError = errors.New("terjadi kesalahan internal pada server")
)
```

### 2. Pemetaan Error ke HTTP Status Code di Layer Handler

Usecase akan mengembalikan error bisnis mentah. Tugas Handler adalah menangkap error tersebut dan memetakan menjadi HTTP Status Code yang tepat:

```
func MapErrorToResponse(err error) (int, string) {
    if errors.Is(err, domain.ErrProductNotFound) {
        return 404, err.Error()
    }
    if errors.Is(err, domain.ErrSKUDuplicate) {
        return 409, err.Error()
    }
    // Default error untuk kegagalan server yang tidak diketahui
}
```

```
    return 500, "Terjadi kesalahan internal pada server"
}
```

### 3. Struktur JSON Response Standar

Buat format JSON yang konsisten untuk sukses dan gagal:

- **Format Sukses:**

```
{
  "success": true,
  "message": "Produk berhasil dibuat",
  "data": { ... }
}
```

- **Format Gagal:**

```
{
  "success": false,
  "message": "Format request tidak valid",
  "errors": ["kolom nama wajib diisi", "harga harus lebih besar dari 0"]
}
```

---

## Sesi 4: Praktikum Mandiri: REST API Produk dengan Clean Architecture & Custom Validation (45 Menit)

Mari kita satukan seluruh konsep Clean Architecture, DTO, custom validator, dan standarisasi error handling ke dalam proyek Go yang terstruktur rapi.

### Struktur Folder Proyek

```
materi-golang/clean-arch/
├── domain/
│   └── product.go
├── repository/
│   └── product_repository.go
├── usecase/
│   └── product_usecase.go
├── delivery/
│   └── product_handler.go
└── main.go
```

## 1. Lapisan Domain: [product.go](#)

```
package domain

import (
    "context"
    "errors"
    "time"
)

var (
    ErrProductNotFound = errors.New("produk tidak ditemukan")
    ErrSKUDuplicate     = errors.New("sku produk sudah terdaftar di sistem")
)

// Entity Bisnis Murni
type Product struct {
    ID          uint
    SKU         string
    Name        string
    Price       float64
    Stock       int
    CreatedAt   time.Time
    UpdatedAt   time.Time
}

// Kontrak Interface untuk Repository
type ProductRepository interface {
    Create(ctx context.Context, product *Product) error
    GetByID(ctx context.Context, id uint) (*Product, error)
    GetBySKU(ctx context.Context, sku string) (*Product, error)
}

// Kontrak Interface untuk Usecase
type ProductUsecase interface {
    Create(ctx context.Context, sku string, name string, price float64, stock int)
    (*Product, error)
    GetByID(ctx context.Context, id uint) (*Product, error)
}
```

## 2. Lapisan Repository: [product\\_repository.go](#)

```
package repository

import (
    "context"
    "errors"
    "time"

    "c_projects/automation/engineer/materi-golang/clean-arch/domain"
)
```

```

        "gorm.io/gorm"
    )

    // GORM DB Model (Hanya ada di layer repository)
    type ProductDB struct {
        ID            uint            `gorm:"primaryKey;autoIncrement"`
        SKU           string          `gorm:"type:varchar(50);uniqueIndex;not null"`
        Name          string          `gorm:"type:varchar(100);not null"`
        Price         float64         `gorm:"type:numeric(12,2);not null"`
        Stock         int             `gorm:"type:int;not null"`
        CreatedAt     time.Time       `gorm:"autoCreateTime"`
        UpdatedAt     time.Time       `gorm:"autoUpdateTime"`
        DeletedAt     gorm.DeletedAt `gorm:"index"`
    }

    // Mengatur nama tabel database GORM
    func (ProductDB) TableName() string {
        return "products"
    }

    type productRepository struct {
        db *gorm.DB
    }

    func NewProductRepository(db *gorm.DB) domain.ProductRepository {
        return &productRepository{db: db}
    }

    func (r *productRepository) Create(ctx context.Context, p *domain.Product) error {
        dbModel := ProductDB{
            SKU:    p.SKU,
            Name:   p.Name,
            Price:  p.Price,
            Stock: p.Stock,
        }

        // Cek duplikasi SKU
        var count int64
        r.db.WithContext(ctx).Model(&ProductDB{}).Where("sku = ?",
p.SKU).Count(&count)
        if count > 0 {
            return domain.ErrSKUDuplicate
        }

        err := r.db.WithContext(ctx).Create(&dbModel).Error
        if err != nil {
            return err
        }

        // Salin ID dan waktu yang digenerate oleh DB ke Entity Domain
        p.ID = dbModel.ID
        p.CreatedAt = dbModel.CreatedAt
        p.UpdatedAt = dbModel.UpdatedAt
    }

```

```

        return nil
    }

    func (r *productRepository) GetByID(ctx context.Context, id uint) (*domain.Product,
    error) {
        var dbModel ProductDB
        err := r.db.WithContext(ctx).First(&dbModel, id).Error
        if err != nil {
            if errors.Is(err, gorm.ErrRecordNotFound) {
                return nil, domain.ErrProductNotFound
            }
            return nil, err
        }

        return &domain.Product{
            ID:          dbModel.ID,
            SKU:         dbModel.SKU,
            Name:        dbModel.Name,
            Price:       dbModel.Price,
            Stock:       dbModel.Stock,
            CreatedAt:   dbModel.CreatedAt,
            UpdatedAt:   dbModel.UpdatedAt,
        }, nil
    }

    func (r *productRepository) GetBySKU(ctx context.Context, sku string)
    (*domain.Product, error) {
        var dbModel ProductDB
        err := r.db.WithContext(ctx).Where("sku = ?", sku).First(&dbModel).Error
        if err != nil {
            if errors.Is(err, gorm.ErrRecordNotFound) {
                return nil, domain.ErrProductNotFound
            }
            return nil, err
        }

        return &domain.Product{
            ID:          dbModel.ID,
            SKU:         dbModel.SKU,
            Name:        dbModel.Name,
            Price:       dbModel.Price,
            Stock:       dbModel.Stock,
            CreatedAt:   dbModel.CreatedAt,
            UpdatedAt:   dbModel.UpdatedAt,
        }, nil
    }
}

```

### 3. Lapisan Usecase: [product\\_usecase.go](#)

```
package usecase
```



```

import (
    "context"

    "c_projects/automation/engineer/materi-golang/clean-arch/domain"
)

type productUsecase struct {
    repo domain.ProductRepository
}

func NewProductUsecase(repo domain.ProductRepository) domain.ProductUsecase {
    return &productUsecase{repo: repo}
}

func (u *productUsecase) Create(ctx context.Context, sku string, name string, price float64, stock int) (*domain.Product, error) {
    // Menjalankan logika bisnis tambahan (misal validasi bisnis)
    newProduct := &domain.Product{
        SKU:    sku,
        Name:   name,
        Price:  price,
        Stock:  stock,
    }

    err := u.repo.Create(ctx, newProduct)
    if err != nil {
        return nil, err
    }

    return newProduct, nil
}

func (u *productUsecase) GetByID(ctx context.Context, id uint) (*domain.Product, error) {
    return u.repo.GetByID(ctx, id)
}

```

#### 4. Lapisan Delivery: [product\\_handler.go](#)

```

package delivery

import (
    "errors"
    "net/http"
    "strconv"

    "c_projects/automation/engineer/materi-golang/clean-arch/domain"
    "github.com/gin-gonic/gin"
)

// DTO Input Request

```

```

type CreateProductRequest struct {
    SKU    string `json:"sku" binding:"required,sku_format"` // Menggunakan custom
validator 'sku_format'
    Name   string `json:"name" binding:"required,min=3,max=100"`
    Price  float64 `json:"price" binding:"required,gt=0"`
    Stock  int     `json:"stock" binding:"required,min=0"`
}

// DTO Output Response (Untuk menyembunyikan kolom sensitif/tidak penting)
type ProductResponse struct {
    ID      uint   `json:"id"`
    SKU     string `json:"sku"`
    Name    string `json:"name"`
    Price   float64 `json:"price"`
    Stock   int     `json:"stock"`
}

type ProductHandler struct {
    usecase domain.ProductUsecase
}

func NewProductHandler(r *gin.Engine, usecase domain.ProductUsecase) {
    handler := &ProductHandler{usecase: usecase}

    // Daftarkan endpoint ke Gin
    r.POST("/api/v1/products", handler.Create)
    r.GET("/api/v1/products/:id", handler.GetByID)
}

func (h *ProductHandler) Create(c *gin.Context) {
    var req CreateProductRequest
    if err := c.ShouldBindJSON(&req); err != nil {
        c.JSON(http.StatusBadRequest, gin.H{
            "success": false,
            "message": "Validasi input gagal",
            "error":   err.Error(),
        })
        return
    }

    // Panggil usecase dengan context
    prod, err := h.usecase.Create(c.Request.Context(), req.SKU, req.Name,
req.Price, req.Stock)
    if err != nil {
        statusCode, errMsg := mapError(err)
        c.JSON(statusCode, gin.H{
            "success": false,
            "message": errMsg,
        })
        return
    }

    // Transformasi domain entity ke DTO Response

```

```

        response := ProductResponse{
            ID:    prod.ID,
            SKU:   prod.SKU,
            Name:  prod.Name,
            Price: prod.Price,
            Stock: prod.Stock,
        }

        c.JSON(http.StatusCreated, gin.H{
            "success": true,
            "message": "Produk berhasil dibuat",
            "data":    response,
        })
    }

    func (h *ProductHandler) GetByID(c *gin.Context) {
        idStr := c.Param("id")
        id, _ := strconv.Atoi(idStr)

        prod, err := h.usecase.GetByID(c.Request.Context(), uint(id))
        if err != nil {
            statusCode, errMsg := mapError(err)
            c.JSON(statusCode, gin.H{
                "success": false,
                "message": errMsg,
            })
            return
        }

        response := ProductResponse{
            ID:    prod.ID,
            SKU:   prod.SKU,
            Name:  prod.Name,
            Price: prod.Price,
            Stock: prod.Stock,
        }

        c.JSON(http.StatusOK, gin.H{
            "success": true,
            "data":    response,
        })
    }

    // Helper untuk standarisasi pemetaan error bisnis ke HTTP Code
    func mapError(err error) (int, string) {
        if errors.Is(err, domain.ErrProductNotFound) {
            return http.StatusNotFound, err.Error()
        }
        if errors.Is(err, domain.ErrSKUDuplicate) {
            return http.StatusConflict, err.Error()
        }
        return http.StatusInternalServerError, "Terjadi kesalahan internal pada

```

```
server"  
}
```

## 5. Perakitan DI dan Menjalankan Server: [main.go](#)

```
package main  
  
import (  
    "strings"  
  
    "c_projects/automation/engineer/materi-golang/clean-arch/delivery"  
    "c_projects/automation/engineer/materi-golang/clean-arch/repository"  
    "c_projects/automation/engineer/materi-golang/clean-arch/usecase"  
    "github.com/gin-gonic/gin"  
    "github.com/gin-gonic/gin/binding"  
    "github.com/go-playground/validator/v10"  
    "gorm.io/driver/postgres"  
    "gorm.io/gorm"  
)  
  
// Daftarkan Custom SKU Validator  
func registerSKUValidator() {  
    if v, ok := binding.Validator.Engine().(*validator.Validate); ok {  
        _ = v.RegisterValidation("sku_format", func(fl validator.FieldLevel)  
bool {  
            sku, ok := fl.Field().Interface().(string)  
            return ok && strings.HasPrefix(sku, "SKU-")  
        })  
    }  
}  
  
func main() {  
    // Inisialisasi Database  
    dsn := "host=localhost user=postgres password=secret45 dbname=ecommercedb  
port=5434 sslmode=disable TimeZone=Asia/Jakarta"  
    db, err := gorm.Open(postgres.Open(dsn), &gorm.Config{})  
    if err != nil {  
        panic(err)  
    }  
  
    // Auto Migration untuk model DB Repository  
    _ = db.AutoMigrate(&repository.ProductDB{})  
  
    // Daftarkan validator kustom  
    registerSKUValidator()  
  
    r := gin.Default()  
  
    // 1. Dependency Injection (DI) - Merakit lapisan dari bawah ke atas  
    productRepo := repository.NewProductRepository(db)  
    productUsecase := usecase.NewProductUsecase(productRepo)
```

```
// 2. Inisialisasi HTTP Handler
delivery.NewProductHandler(r, productUsecase)

// Jalankan server
r.Run(":8080")
}
```

---

## Sesi 5: Soal Latihan Praktik E-Commerce (Mandiri)

---

Selesaikan dua latihan pemeliharaan arsitektur di bawah ini untuk memperkuat pemahaman Anda mengenai standarisasi Clean Architecture dan DTO di Go.

### Latihan 1: Validasi Format Email Domain Perusahaan (Custom Validator)

**Deskripsi Soal:** Dalam sistem registrasi merchant e-commerce internal, perusahaan membatasi bahwa semua merchant wajib menggunakan email resmi berakhiran domain perusahaan (contoh: @merchant.juaracoding.com ).

#### Kriteria Implementasi:

1. Buatlah custom validator bernama `merchant_email` . Fungsi validator ini harus memvalidasi bahwa string email wajib berakhiran `@merchant.juaracoding.com` .
2. Terapkan validator ini pada struct DTO Registrasi Merchant baru.
3. Uji dengan payload email yang valid dan tidak valid, lalu pastikan error response mengembalikan HTTP 400 Bad Request dengan pesan eror yang seragam.

---

### Latihan 2: Implementasi Layanan Fitur Profil Customer (Clean Architecture)

**Deskripsi Soal:** Bangunlah satu domain baru bernama **Customer** mengikuti prinsip Clean Architecture dengan detail sebagai berikut.

#### Kriteria Implementasi:

1. **Domain Layer:**
  - Entity `Customer` (ID, Name, Email, Address).
  - Interface `CustomerRepository` dan `CustomerUsecase` .
2. **Repository Layer:**
  - Struct DB model `CustomerDB` dengan tablename `"customers"` .
  - Implementasi method `GetByEmail(ctx, email)` dan `Create(ctx, customer)` .
3. **Usecase Layer:**

- Logika pendaftaran customer baru. Jika email customer sudah terdaftar, kembalikan sentinel error `ErrEmailRegistered` .

#### 4. **Delivery Layer:**

- Endpoint `POST /api/v1/customers` yang menerima DTO request input (Name, Email, Address).
- Lakukan standarisasi error handling: Jika email terduplikasi, kembalikan HTTP 409 Conflict.